

AI Principles and Practice · COMP-0484

Unsupervised Learning and the Idea of a Representation

Week 5 · Reference Booklet

A plain-English guide: finding structure without labels, and the geometry that carries meaning

Lecturer: Jonathan Dunne PhD

How to use this booklet

This booklet is the long, patient companion to the Week 5 lecture. Where the slides carry the argument and the lab puts it into practice, this document slows everything down: the full definitions, the mathematics written out and explained in words, and the worked example laid bare so you can reproduce every number for yourself. Read it after the lecture, and keep it open during the lab.

The example running through every section is the same credit dataset you audited for fairness in Week 3 and stress-tested in Week 4. This week we do something new to it: we throw away the repayment label, because unsupervised learning has no answer to predict, and we throw away the `group` column too, as we always do. Then we let two methods, clustering and PCA, tell us what shape the applicants make. Following one dataset across three weeks is deliberate. The same proxy leakage that made the model unfair in Week 3 is what, this week, lets a label-free method reconstruct the protected group from geometry alone. Seeing that happen on data you already know is the fastest route to the single idea of the week: a representation is a learned description of your data, and it is never neutral.

Setting up the tools (once)

You need Python and the same five libraries as before. Every figure and every printed number in this booklet is built with only those five; there is nothing extra to install this week.

terminal

```
python -m venv .venv && source .venv/bin/activate
pip install numpy pandas scikit-learn scipy matplotlib
python week05-make_assets.py          # recreates every figure and printed number
```

What the libraries are for

NumPy does fast maths on arrays. **pandas** holds the table of applicants. **scikit-learn** supplies the scaler, K-means, the Gaussian mixture, PCA, the silhouette machinery, and the small probe we use to measure leakage. **SciPy** provides the hierarchical clustering and the dendrogram. **Matplotlib** draws every chart. That is the whole toolkit; the power this week is in the ideas, not the imports.

Contents

How to use this booklet	2
Setting up the tools (once)	2
1 What “unsupervised” actually means	4
1.1 Two kinds of learning	4
1.2 Proposing structure, not discovering it	4
2 Clustering I: K-means and the assumptions it hides	5
2.1 The algorithm	5
2.2 The objective it is really minimising	5
2.3 Why standardisation is not optional	5
2.4 The three assumptions you agree to	6
3 Choosing K: a decision you defend	7
3.1 The elbow method	7
3.2 The silhouette	7
3.3 When the two tools disagree	7
3.4 Reading the silhouette per point	8
4 Clustering II: hierarchical and Gaussian mixtures	9
4.1 Hierarchical (agglomerative) clustering	9
4.2 Gaussian mixture models	9
4.3 Three algorithms, three theories	10
5 The limits of cluster validity	11
6 Dimensionality reduction and PCA	12
6.1 The curse of dimensionality	12
6.2 The idea of PCA	12
6.3 The mathematics, in words	12
6.4 How much you keep, and how much you throw away	12
6.5 Reading the axes: loadings and the biplot	13
7 PCA as your first learned representation	14
8 Where PCA fails: the straight-axis limit	15
9 Embeddings: geometry as meaning	16
9.1 From components to embeddings	16
9.2 Distance and direction	16
9.3 The representation is the product	16
9.4 Linear now, nonlinear soon	16
10 Representations encode bias	18
10.1 The reveal	18
10.2 Put a number on it	18
10.3 Why it happens	18
10.4 A representation is never neutral	19
11 The lab, step by step	20
11.1 Step 1: cluster and validate	20
11.2 Step 2: reduce, project and name the axes	20
11.3 Step 3: interrogate the representation	20
11.4 What to hand in	21
12 Key terms	22

1 What “unsupervised” actually means

For four weeks you have done supervised learning, whether or not we called it that. Someone handed you a labelled answer, and your job was to reproduce it on data you had not seen. This week the label is gone, and its absence changes the problem more deeply than it first appears.

1.1 Two kinds of learning

Supervised learning you are given pairs of inputs and correct answers, and you learn a rule that reproduces the answer on new inputs. Success has a precise definition, and a test set to measure it.

Unsupervised learning you are given inputs only, with no answers, and you look for structure: groups, directions, a lower-dimensional shape. There is no correct answer to reproduce, and so nothing that can mark your output right or wrong.

That second point is the one to sit with. The label did a great deal of quiet work in supervised learning. It gave you accuracy, a stopping rule, a way to compare two models, a way to detect overfitting. Remove it and all of that machinery leaves with it. You are no longer being graded against a hidden answer key, because there is no answer key.

1.2 Proposing structure, not discovering it

The most common misconception about clustering is baked into the way we talk about it. We say a method “finds the groups that are really there”, as if the groups pre-exist and the algorithm merely locates them. Unlearn that phrase. A clustering algorithm imposes a structure of your choosing, so many groups, of a particular assumed shape, and then reports how well the data tolerated that imposition. The output is not a fact you discovered. It is a hypothesis you proposed, and the burden now falls on you to argue that it is a useful one.

The shift in one sentence

Supervised learning asks “can I reproduce the known answer?”. Unsupervised learning asks “is this proposed structure worth believing?”, and that second question can never be settled by a single number. It ends in a judgement.

This is why the whole first half of the week is really about evaluation and defence, not about running algorithms. Running K-means is one line. Deciding whether its output means anything, and being able to say why, is the actual skill.

2 Clustering I: K-means and the assumptions it hides

K-means is the first clustering algorithm anyone learns, because it is fast, scalable and short to describe. Those virtues hide a set of strong assumptions that you accept silently every time you run it. Understanding those assumptions is more important than memorising the steps.

2.1 The algorithm

K-means partitions your points into a number of clusters, K , that you fix in advance. It then repeats a simple two-step loop, known as Lloyd's algorithm, until the assignments stop changing.

1. **Assign:** give each point to the cluster whose centre (the “centroid”) is nearest, measured by straight-line distance.
2. **Update:** move each centroid to the mean of the points currently assigned to it.

Starting from a set of initial centres, these two steps chase each other until nothing moves. Because the starting centres are random, the result can vary between runs, which is why in practice we run it several times and keep the best; the `n_init` argument in scikit-learn does exactly this.

2.2 The objective it is really minimising

The loop is not arbitrary. It is descending a specific quantity, the within-cluster inertia, which is the total squared distance from every point to its own centroid.

$$W = \sum_{c=1}^K \sum_{x \in C_c} d(x, \mu_c)^2$$

Within-cluster inertia. C_c is cluster c , μ_c its centroid, and d the Euclidean distance below. Smaller W means tighter clusters.

The distance itself is the ordinary straight-line, or Euclidean, distance between two points described by p features.

$$d(x, y) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2}$$

Euclidean distance in p dimensions. Every squared feature difference contributes, so features on larger scales contribute more.

Read those two formulas together and a warning falls out of them. Inertia is a sum of squared feature differences, so a feature measured on a large numerical scale will dominate the sum, regardless of whether it is the most meaningful feature. That is not a detail; it decides your clusters.

2.3 Why standardisation is not optional

In our credit data, income runs in the tens of thousands and the debt ratio sits between zero and one. Put them side by side in the distance formula and income's magnitude swamps everything: a difference of a few thousand pounds in income dwarfs the entire range of the debt ratio. The clustering becomes, in effect, a sort on income alone, with the other three features contributing almost nothing.

The fix is to standardise every feature to a common scale before clustering, by subtracting its mean and dividing by its standard deviation.

$$z = \frac{x - \bar{x}}{s}$$

Standardisation. Each feature is recentred to mean zero and rescaled to standard deviation one, so all features get a comparable vote.

Standardisation is therefore not a cosmetic step you do because a textbook told you to. It silently decides which features influence the result. Change it, and you change the clusters. Whenever a clustering looks suspiciously like a sort on one column, suspect the scaling before you believe the finding.

2.4 The three assumptions you agree to

Behind that short algorithm sit three commitments. Every time you call K-means, you are asserting all three about your data.

Round, equal clusters because it uses plain Euclidean distance to a centroid, K-means expects clusters that are roughly spherical and of similar spread. Long, thin or unequal clusters confuse it.

Hard membership every point belongs fully to exactly one cluster. There is no notion of a borderline case that half-belongs to two.

Scale sensitivity “near” means Euclidean distance, so the scaling of your features is part of the model, not a preprocessing afterthought.

The assumption is the model

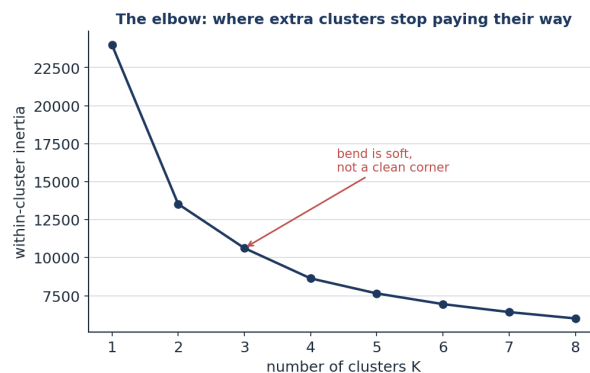
K-means is not neutral machinery you aim at data. It is a specific claim that your data is a handful of equal round balls. When that claim is wrong, and on real data it is usually at least a little wrong, the algorithm does not warn you. It returns a clean, confident partition that quietly misreads the shape. A method that always looks certain is one you must interrogate rather than trust.

3 Choosing K: a decision you defend

K-means makes you choose the number of clusters before it will say anything, and there is no true K hidden in the data waiting to be read off. K is an argument you make. Two tools inform that argument. Neither settles it.

3.1 The elbow method

Plot the within-cluster inertia W as K increases. It always falls, because more centres always fit the data more tightly; taken to the extreme, K equal to the number of points drives W to zero and teaches you nothing. So you do not seek the minimum. You seek the bend, or “elbow”, the value of K after which each additional cluster buys you very little extra tightness.



Inertia against K on the credit data. It falls smoothly from about 13,530 at $K=2$ to about 6,000 at $K=8$. There is a gentle bend near three, but no sharp corner.

On our applicants, inertia runs 13,530 at $K = 2$, 10,630 at $K = 3$, 8,638 at $K = 4$, and keeps sliding down to about 6,000 by $K = 8$. There is a soft bend around three, but nothing you could honestly call a corner. This is the normal state of real data, not a defect of ours. Populations arrive as continuous smears with soft concentrations, not as crisply separated balls, so a clean elbow is the exception. If you are ever shown a perfect right-angle elbow, ask whether the data was simulated.

3.2 The silhouette

The silhouette takes a different angle: for each point it asks whether that point sits comfortably in its own cluster, or whether it would be happier next door.

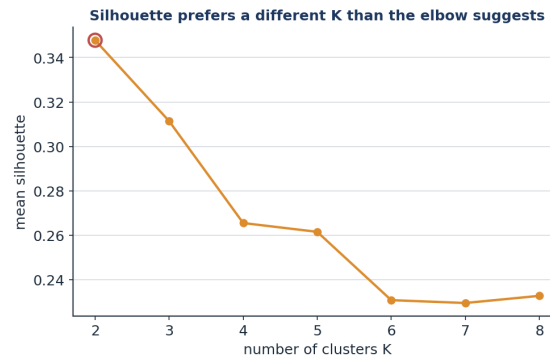
$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

For point i : $a(i)$ is its mean distance to its own cluster, $b(i)$ its mean distance to the nearest other cluster. Values run from -1 to $+1$.

A silhouette near $+1$ means the point is far tighter to its own cluster than to any other: a clean, well-placed point. Near 0 means it sits on the border, and the assignment was almost a coin toss. Below 0 means it is actually closer to a neighbouring cluster than its own: a probable misassignment. Average $s(i)$ over all points and you get a single score for the whole clustering.

3.3 When the two tools disagree

On our data they disagree, and that disagreement is the lesson, not an accident.



Mean silhouette against K . It peaks at $K=2$ (0.35) and declines steadily. Where the elbow hinted at three or four, the silhouette prefers two.

The mean silhouette is highest at $K = 2$, at 0.348, and falls away as K grows: 0.311 at three, 0.265 at four, and downhill from there. So the elbow's soft hint towards three or four is contradicted by the silhouette's clear preference for two. Two respected internal measures, computed on one dataset, rank the options differently, because each rewards a different virtue: the elbow rewards tightness, the silhouette rewards separation.

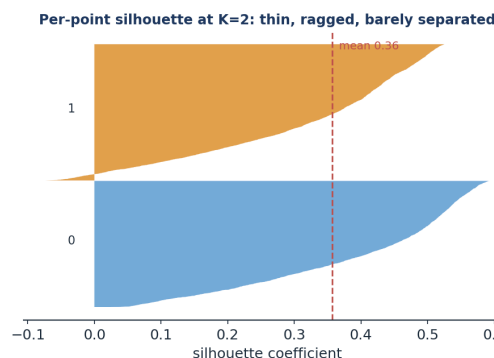
Notice the height as well as the peak. A silhouette of 0.35 is not strong. It says the two-cluster structure is real but soft, with a substantial number of points sitting near the border between clusters. The honest conclusion is not “the answer is two” but “if I must choose, two is the more defensible call, and here is why, while acknowledging the structure is weak”.

What you are actually assessed on

There is no rule that resolves an elbow-versus-silhouette disagreement for you. Do not write code that silently picks the higher silhouette and moves on. Print both, look at them, choose a K , and defend the choice in a sentence. The reasoning is the deliverable; the loop is plumbing.

3.4 Reading the silhouette per point

The averaged silhouette hides its own distribution, and averages mislead by omission. The per-point “knife” plot restores the detail: one horizontal sliver per applicant, grouped by cluster and sorted from worst to best within each.



Per-point silhouette at $K=2$. The blades are thin and uneven, with a long tail approaching zero: the fence-sitters the mean of 0.35 quietly absorbed.

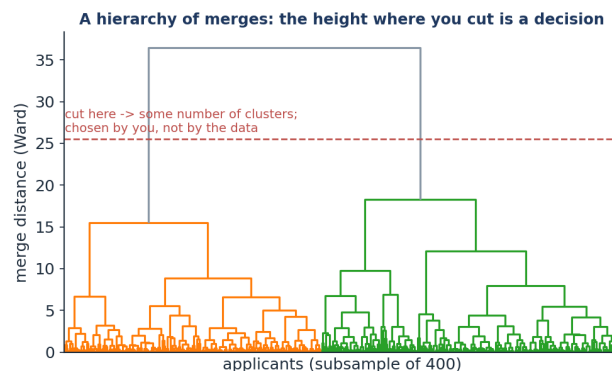
Wide, full blades signal a cohesive cluster whose members are comfortable. Thin, ragged blades, or bars dipping below zero, mark points that would sit more happily elsewhere. On our data the blades are thin with a long low tail, which is the visual form of that unimpressive 0.35. The general habit is worth keeping well beyond clustering: a healthy average can conceal a sick component, so look at the shape before you quote the summary number.

4 Clustering II: hierarchical and Gaussian mixtures

K-means is one theory of what a cluster is. Two other methods encode different theories, and comparing their assumptions is more useful than collecting their pseudocode.

4.1 Hierarchical (agglomerative) clustering

Where K-means demands K up front, hierarchical clustering refuses to choose it and makes the refusal visible. It begins with every point as its own cluster, then repeatedly merges the two closest clusters, recording each merge, until everything is joined into one. The record is a tree, the dendrogram.



A dendrogram on 400 applicants, built with Ward linkage. The height of each join is the cohesion it cost. Cut the tree horizontally to obtain clusters; where you cut is your decision.

The height of each join measures how reluctant the merge was: low joins fuse very similar points, tall joins fuse groups that were quite different. To turn the tree into clusters you slice it horizontally. Cut low for many tight clusters; cut high for a few loose ones. The algorithm builds the whole hierarchy, but hands you the scissors, so the number of clusters is again your choice, made explicit rather than hidden.

Two knobs change the tree, and you should interrogate both rather than accept defaults. The **distance** defines how far apart two points are; the **linkage** defines how far apart two clusters are (Ward linkage, used here, merges the pair that increases total inertia least). Change either and the same data grows a different tree.

4.2 Gaussian mixture models

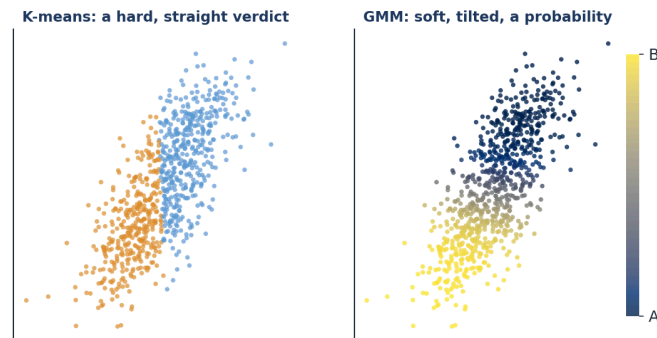
A Gaussian mixture model, or GMM, relaxes the two rigidities that most often trip up K-means. It models the data as a blend of K Gaussian “bells”, each with its own centre, its own elliptical shape and tilt, and its own weight.

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

The mixture density. π_k is the weight of component k , μ_k its centre, and Σ_k its covariance, which sets the ellipse’s shape and orientation.

Two things follow from that covariance term Σ_k . First, clusters can be tilted ellipses of unequal spread, not just equal circles. Second, membership becomes soft: rather than a hard verdict, the model returns

the probability that a point came from each component. An applicant might be seventy-eight per cent one cluster and twenty-two per cent another.



Two tilted, overlapping blobs. K-means (left) slices them with a straight boundary and mislabels the overlap; the GMM (right) recovers the tilt and expresses the overlap as graded probability.

K-means is in fact the special case of a GMM in which you force every covariance to be an equal sphere and round every probability to zero or one. The soft assignment is not a nicety: the applicants sitting near fifty-fifty are exactly the borderline cases a human should review, and a hard label would have hidden them. Because a GMM has a proper likelihood, you can also compare different K values with an information criterion such as the BIC, which rewards fit while penalising extra parameters, giving a more principled model-selection route than the elbow.

4.3 Three algorithms, three theories

	K-means	Hierarchical	GMM
Shape assumed	equal spheres	set by linkage	tilted ellipses
Assignment	hard	hard, set by cut	soft probabilities
You must choose	K up front	where to cut	K and covariance form
Fails when	clusters non-round or unequal	one early bad merge propagates	wrong K invents ellipses

The single most important property they share is the uncomfortable one: each will return an answer on any data at all, including data with no clusters. Hand K-means uniform noise and ask for three clusters and you will get three, tidy and confident. The algorithm cannot tell you whether structure exists; only whether the data tolerated the structure you imposed. The burden of showing the clusters are real rests on you.

5 The limits of cluster validity

Because there is no label, “is this clustering good?” cannot be answered the way “is this classifier accurate?” can. There are two families of validity measure, and it is vital to know what each can and cannot see.

Internal validity measures such as the silhouette and inertia judge the geometry alone, are the clusters tight and separated. Their advantage is that they need no ground truth. Their blindness is that they say nothing about whether the clusters mean anything.

External validity measures that compare your clusters to known labels. They speak to meaning, but they need labels you usually do not have; if you had them, you might not be clustering.

The trap to remember

A high silhouette certifies a tidy partition. It says nothing about whether that partition is useful, fair, or merely an artefact of your scaling. A clustering can be geometrically excellent and completely meaningless, and, as Section 8 shows on our own data, it can be geometrically excellent and quietly discriminatory while the silhouette stays perfectly content.

The consequence is that the last word in unsupervised learning is never a metric. It is a human, reading the clusters against the domain and judging whether they earn their keep. Every internal number is an input to that judgement, not a replacement for it.

6 Dimensionality reduction and PCA

Clustering leaned on distances in four dimensions that we could not see. Before trusting them further, we should ask whether distance behaves in high dimensions, and then meet the method that lets us compress and, for the first time, watch a machine learn a new description of the data.

6.1 The curse of dimensionality

Our four features are gentle; real feature sets run to dozens or hundreds, and dimensions are not free. As you add them, the volume of the space grows so fast that points spread out until almost everything is roughly equidistant from everything else. The relative gap between your nearest and farthest neighbour shrinks towards nothing, so “nearest” loses its force, and every distance-based method, including all of the clustering above, quietly weakens. This is the curse of dimensionality, and it is the normal condition of real data, not a corner case. The natural response: if most of the story lives in a few directions, find those directions and drop the rest.

6.2 The idea of PCA

Your features arrived with axes chosen by whoever built the spreadsheet, and there is no reason the data’s real variation lines up with them. Principal Component Analysis throws those axes away and asks a better question: along which single direction does the data vary the most? That direction is the first principal component, PC1. The next most variable direction at right angles to it is PC2, and so on. Keep the first few and you retain most of the variation in far fewer numbers, expressed on axes the data itself selected.

6.3 The mathematics, in words

PCA is an eigenvalue problem on the covariance matrix of the centred, standardised data.

$$C = \frac{1}{n-1} X^\top X$$

The covariance matrix of the centred data X . Entry (i, j) measures how features i and j vary together.

The principal components are the eigenvectors of C , and each one’s eigenvalue is the amount of variance along it.

$$C\mathbf{v}_j = \lambda_j \mathbf{v}_j$$

Eigenvector $\mathbf{v}_j$ is the direction of the j -th component; its eigenvalue λ_j is the variance captured along that direction.

To reduce the data to k dimensions, you project it onto the top k eigenvectors, collected as columns of a matrix V_k .

$$Z = XV_k$$

The reduced representation. Each row of Z is one applicant, now described by k coordinates on the learned axes instead of p raw features.

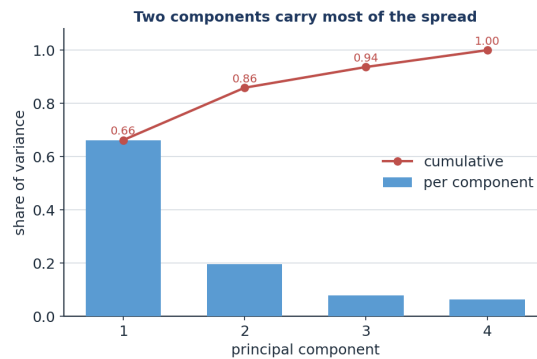
You never have to do this by hand; scikit-learn’s PCA computes it in one line. What matters is the reading: PCA rotates the coordinate system to align with the directions of greatest variance, and hands back a shorter description of each point.

6.4 How much you keep, and how much you throw away

Each eigenvalue, expressed as a fraction of the total, is the share of variance one component explains.

$$\text{explained variance ratio}_j = \frac{\lambda_j}{\sum_{i=1}^p \lambda_i}$$

The proportion of total variance captured by component j . Summed over the kept components, it tells you how much of the data's spread survives the reduction.

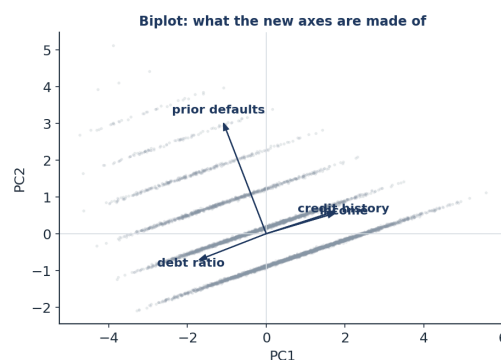


The scree plot on our data. PC1 carries 66% of the variance, PC2 a further 20%; together the first two axes hold about 86%, leaving the last two with the scraps.

On the credit data, PC1 alone captures 66.2% of all variation and PC2 a further 19.7%, so two axes hold 85.9% of everything; PC3 and PC4 carry the remaining 7.8% and 6.3%. Reducing four features to two therefore costs about fourteen per cent of the variance and buys a picture you can actually look at. Whether that trade is worthwhile depends on your problem, not on a rule. Note the assumption you accept by keeping the high-variance components: that variance is where the signal lives. Often true, but not always; a rare, low-variance direction can be exactly the fraud or fault you were hired to find, and PCA discards it first.

6.5 Reading the axes: loadings and the biplot

A reduced plot with axes named PC1 and PC2 is unfinished. Each component is a weighted blend of the original features, and those weights, the loadings, tell you what the axis is made of.



A biplot. The arrows are the feature loadings on PC1 and PC2: income and credit history pull one way, debt ratio and prior defaults the other.

On our data, PC1's loadings are income +0.549, credit history +0.547, debt ratio −0.533 and prior defaults −0.339. Read that and PC1 is unmistakable: high income and long history at one end, high debt and past defaults at the other. That is not “PC1”; it is a **financial-health axis** the data drew for itself. Translating a vector of loadings into a sentence a domain expert would accept, “PC1 is a creditworthiness axis, driven positively by income and history and negatively by debt”, is the interpretive skill this week assesses. Leave the axes unnamed and you have decorated, not interpreted.

7 PCA as your first learned representation

Step back from eigenvectors and look at what happened. PCA took raw features and returned a new coordinate system, fitted to this data, in which each applicant is now described by a short vector on axes the algorithm invented. The applicant did not change; their description did. That learned description is what we mean by a representation.

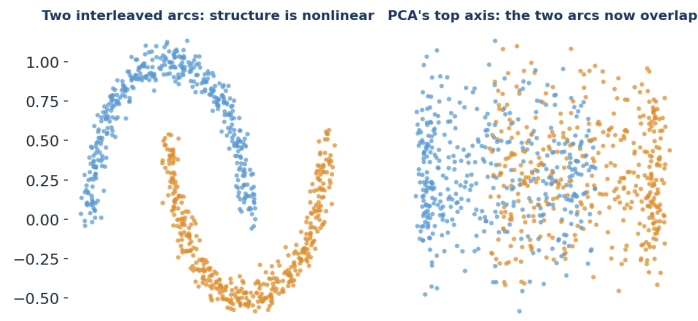
The sentence to carry

A representation is a learned answer to the question “how should this data be described?”. PCA is the simplest honest example: no human chose its axes, the covariance of the data did. This is why PCA belongs at the centre of the week, not filed away as a compression trick.

The reason it matters reaches far beyond PCA. Everything downstream now runs on the representation rather than the raw data: the clustering, the visualisation, any model you build next. They all see the two coordinates, not the original four features. So the choice of representation quietly shapes everything that follows it. Change the representation and you change what every later step can even see. From here on, a reflex question for any system is “what representation is this running on, and what did whoever built it bake in?”.

8 Where PCA fails: the straight-axis limit

PCA is linear: its axes are straight lines. When the true structure of the data curves, a straight projection can flatten it into nonsense.



Two interleaved arcs (left) are cleanly separable, but not by a straight line. Projected onto PCA's top axis (right), the two classes land on top of each other.

The two interleaved arcs on the left are obviously separable to the eye, but by no straight line and along no single straight axis. Project them onto the first principal component, the direction of greatest variance, and the classes overlap completely. The variance was real; it simply was not the signal. Evaluate PCA by its assumption, not its popularity: when it fails, it fails quietly, returning a confident low-dimensional picture that has discarded the structure you cared about. This exact gap, curved structure that straight axes cannot hold, is the reason nonlinear methods and learned embeddings exist, which is where the biggest idea of the week arrives.

9 Embeddings: geometry as meaning

PCA gave every applicant a short vector on learned axes. Loosen three of its requirements and you arrive at the idea the rest of the course is built on.

9.1 From components to embeddings

Drop the requirement that the axes be straight. Drop the requirement that there be only a few. Drop the requirement that they be human-nameable. What remains, a learned map from each item to a point in some space, is the general idea of an embedding. An embedding places each item, an applicant, a word, an image, a molecule, as a point in a space some method learned, arranged so that position carries meaning. PCA is the simplest honest example; everything more powerful later in the course is a richer version of the same move.

9.2 Distance and direction

Once your items are points in a learned space, the geometry of that space is your model of similarity, and two pieces of geometry do the work.

Distance as similarity points near each other are things the representation treats as alike. Nearest-neighbour search, recommendation and retrieval all reduce to measuring distance in a well-built space.

Direction as relationship in a good embedding, consistent offsets encode consistent relationships. The classic example from word embeddings is that “king” minus “man” plus “woman” lands near “queen”: the direction meaning “male to female” is a consistent direction in the space, learned rather than programmed.

The idea the module turns on

A representation is a space you have arranged so that the questions you care about become geometry. Get the space right, and a hard question, is this like that, what relates to this, becomes something you answer by measuring a distance or following a direction. You met the simplest form of this today, in PCA.

9.3 The representation is the product

It is tempting to think the model is the clever part and the representation is mere preparation. Modern systems invert that: most of their effort goes into learning a good representation, and the task-specific layer on top is often small. A model almost never reasons over raw pixels or characters; it reasons over an embedding, and the quality of that embedding sets a ceiling on everything above it.

9.4 Linear now, nonlinear soon

PCA is the linear anchor. Later methods trade its guarantees for reach, and each trade costs something worth knowing.

	What it buys	What it costs
PCA (linear)	fast, interpretable axes, faithful global structure	misses curved structure
t-SNE / UMAP	beautiful local neighbourhoods, clusters pop out	distances between clusters can be meaningless
Learned embeddings	task-tuned, richly expressive	opaque axes, need data and compute, hard to audit

A caution you will need

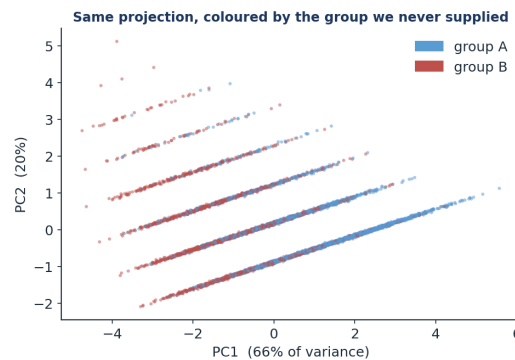
With t-SNE and UMAP, do not read the gaps between clusters as real distances. Those methods optimise local neighbourhoods and will stretch or squash the space between groups to make a tidy layout, so “these two clusters are far apart in the plot” is often meaningless. Critique the method before you narrate its picture.

10 Representations encode bias

Everything so far has been machinery. Here is where it collects on the promise made in the lecture: an unsupervised representation, built with no labels and no protected column, reconstructs the protected group anyway.

10.1 The reveal

Take the two-dimensional PCA map of the applicants, exactly as computed, and change one thing only: colour each point by the `group` column we deleted before any algorithm ran.



The same PCA projection, recoloured by the protected group we never supplied. The groups separate, largely along PC1, the financial-health axis the data drew for itself.

The groups do not blend together. They separate, and mostly along PC1, the very financial-health axis PCA built from income, history, debt and defaults. We removed the group column and used no labels, yet the representation has laid the group out in space.

10.2 Put a number on it

A picture invites argument, so measure it. Fit a small model that sees only PC1, one number per applicant, and ask it to name each person's group.

Quantity	Value
Group recoverable from PC1 alone	0.711
Base rate (guess the majority group)	0.426
For comparison: Week 4, group recoverable from the supervised score	0.70

PC1 alone recovers the protected group 71.1% of the time, against a base rate of 42.6%. Read the last row beside it: last week the **supervised** credit score leaked the same group at about 70%. An unsupervised representation, built with no labels and no group column, encodes the protected attribute at almost exactly the rate the supervised model did. Removing the column removed the word, not the information.

A useful nuance sits underneath that headline. The hard two-way K-means split matches the group only weakly (an adjusted Rand index of 0.156, and a purity of 0.701), so the **clusters** are not the group. It is the continuous axis, PC1, that carries the group so strongly. Distinguishing “the clusters roughly track it” from “one learned axis recovers it almost fully” is exactly the kind of careful reading the lab rewards.

10.3 Why it happens

This is the proxy leakage of Weeks 3 and 4 in geometric form, and it is arithmetic, not sorcery. The world that produced the data is unequal, so the groups differ in income, history and debt; those

features are therefore correlated with group; the largest axis of variation in the data runs along the direction the groups differ; and PCA, which by construction hunts the largest axis of variation, finds that direction and names it PC1. PCA never sought the group, it has no concept of group, it sought variance, and in this data the biggest source of variance is the inequality itself. No malice, no bug, no protected column: just correlation, faithfully compressed. That is what makes the failure mode dangerous, it arrives by default, in well-intentioned hands, on clean code.

10.4 A representation is never neutral

The principle

It is tempting to treat an unsupervised method as objective, with no labels and no human choices. That framing is the danger. A representation inherits every correlation in the data, including the ones you designed out in Week 3, and it cannot tell fair correlations from harmful ones. The fairness question does not close when you drop the sensitive attribute; it reopens the moment you learn a representation, because the representation can rebuild that attribute and pass it silently downstream. “We removed that field” is a statement about your schema, not your geometry.

And the metrics from earlier cannot save you here. Our clustering scored a perfectly respectable silhouette, which certified geometric tidiness and was structurally incapable of noticing that the tidy split ran along a protected group. Internal measures see shape, not justice; a clustering can be cohesive, well separated, high-scoring and discriminatory at once, and no silhouette will raise a hand. This is the concrete, numerical reason the last word belongs to a human reading the representation against the harm it could do.

11 The lab, step by step

The lab runs the whole argument on the credit applicants, in four steps and in order. The deliverable is an interpretation, not a gallery of plots.

11.1 Step 1: cluster and validate

Standardise first, then loop over candidate values of K , printing both the inertia and the silhouette for each. Look at the two columns yourself and choose a K you can defend.

```
cluster.py

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

Xs = StandardScaler().fit_transform(X)      # scale FIRST

for k in range(2, 9):
    km = KMeans(k, n_init=10, random_state=7).fit(Xs)
    print(k, round(km.inertia_, 1),
          round(silhouette_score(Xs, km.labels_), 3))
```

K, inertia, silhouette (our data)

```
2 13530.2 0.348    <- silhouette prefers this
3 10629.9 0.311
4  8637.9 0.265    elbow soft; the two disagree
```

11.2 Step 2: reduce, project and name the axes

Fit PCA, read the explained-variance ratio, project to two dimensions for a picture, and, crucially, print PC1's loadings so you can name the axis in domain terms.

```
reduce.py

from sklearn.decomposition import PCA

pca = PCA().fit(Xs)
print(pca.explained_variance_ratio_)      # -> [0.66, 0.20, 0.08, 0.06]

Z = PCA(n_components=2).fit_transform(Xs)  # your learned 2D representation

for name, w in zip(features, pca.components_[0]):
    print(name, round(w, 3))              # read PC1's recipe
```

PC1 loadings (our data)

```
income          +0.549
credit history  +0.547
debt ratio      -0.533    -> a "financial health" axis
prior defaults  -0.339
```

11.3 Step 3: interrogate the representation

This is the step that makes the lab more than a clustering exercise. Attack your own map: if a one-feature probe can recover a sensitive attribute from PC1, your representation is a proxy for it, and you must say so plainly.

`leak.py`

```
from sklearn.linear_model import LogisticRegression

pc1 = Z[:, [0]] # the top axis alone
probe = LogisticRegression().fit(pc1, sensitive)
print("recoverable:", round(probe.score(pc1, sensitive), 3))
print("base rate :", round(sensitive.mean(), 3))

# then LOOK: recolour the 2D scatter by the sensitive attribute
# plt.scatter(Z[:,0], Z[:,1], c=sensitive, s=8)
```

11.4 What to hand in

- A defended choice of K, with the elbow and silhouette shown and one sentence justifying the value you picked.
- A two-dimensional PCA map with named axes, backed by the loadings, not left as PC1 and PC2.
- A leakage check on any sensitive attribute you hold, with the recovery rate and base rate stated plainly, and a sentence on what it means for any downstream use.

Strong interpretation versus box-tick

A box-tick report says “we found 3 clusters; PCA explained 86%; here is a plot”. A strong report names what each cluster is in the domain and which are shaky, reads PC1 as a financial-health axis with the loadings as evidence, reports that PC1 recovers the group at 0.71 and treats that as a finding, and states which structure it would refuse to act on and why. The plot is the easy fifth of the work; the grade lives in the interpretation.

This exploratory pass is not throwaway: the same four moves return as Elective 2 of your assignment, run there on the dataset the team assigns you, and the leakage result feeds directly back into the Week 3 fairness work and the Week 4 governance note. Treat the credit data as your rehearsal; the marks are for the argument you build on top of the plots, not for the plots themselves.

12 Key terms

Unsupervised learning learning structure from inputs with no labelled answers; there is no accuracy and no test set, only a proposed structure to defend.

Cluster a group of points a method treats as belonging together; a proposal about the data, not a fact discovered in it.

Inertia total squared distance from each point to its cluster centroid; the quantity K-means minimises, and the basis of the elbow.

Silhouette a score from -1 to $+1$ measuring how much better a point fits its own cluster than the nearest rival; averaged, a measure of separation.

Linkage in hierarchical clustering, the rule for the distance between two clusters (for example Ward), which shapes the dendrogram.

Gaussian mixture model (GMM) a soft clustering that models the data as weighted tilted Gaussians and returns membership probabilities.

Dimensionality reduction replacing many features with a few that retain most of the structure, so distances become meaningful and data becomes visualisable.

Principal Component Analysis (PCA) a linear reduction that rotates the axes to the directions of greatest variance; your first learned representation.

Loadings the weights that combine original features into a principal component; reading them is how you name the axis.

Explained variance ratio the share of total variance captured by a component; guides how many components to keep.

Representation a learned description of data, for example the coordinates PCA assigns; never neutral, since it inherits every correlation in the data.

Embedding a learned map placing items as points in a space where distance and direction carry meaning; the idea unifying the course.

Proxy leakage the reconstruction of a removed sensitive attribute from correlated features that remain; here, visible as separation along PC1.